

Appunti di Algoritmi e Complessità

per il corso di Boldi P. @ Università degli studi di Milano

Manuele Ferro

2026 / 2027

Indice

1	Introduzione	2
2	Analisi di algoritmi	2
2.1	Load balancing	2

1 Introduzione

2 Analisi di algoritmi

2.1 Load balancing

Problema:

Input:

Ho in input un numero $n \in \mathbb{N}^+$ di task e un numero $m \in \mathbb{N}^+$ di macchinari.

Ognuna di queste task ha un tempo di esecuzione $t_0, t_1, \dots, t_{n-1} \in \mathbb{N}^+$

Indichiamo con L_i il carico associato a una macchina i .

Indichiamo con L il costo, definito come $L = \max_i L_i$

Soluzioni ammesse:

Le soluzioni ammesse per questo problema sono delle partizioni di n per m :

$$A_0, A_1, \dots, A_{m-1}$$

Il problema è di ricerca del minimo.

Esempio grafico:

Supponiamo di avere 8 task, e 3 macchinari su cui eseguirle.

Tasks:

0	1	2	3	4	5	6	7
3	2	3	1	1	4	5	1

Possiamo trovare una possibile partizione delle task per i macchinari del tipo:

m	tasks	L_i				
	0 1 2 3					
0	<table><tr><td>3</td><td>1</td><td>1</td><td>2</td></tr></table>	3	1	1	2	7
3	1	1	2			
	0 1 2					
1	<table><tr><td>5</td><td>1</td><td>3</td></tr></table>	5	1	3	8	
5	1	3				
	0 1					
2	<table><tr><td>4</td><td>1</td></tr></table>	4	1	5		
4	1					

Vediamo facilmente che il costo L è il massimo tra i 3, ovvero 8. Possiamo allo stesso tempo affermare che non è la soluzione ottima, infatti, è semplice evidenziarne una migliore.

m	tasks	L_i			
	0 1 2				
0	<table><tr><td>5</td><td>1</td><td>1</td></tr></table>	5	1	1	7
5	1	1			
	0 1				
1	<table><tr><td>4</td><td>3</td></tr></table>	4	3	7	
4	3				
	0 1 2				
2	<table><tr><td>3</td><td>2</td><td>1</td></tr></table>	3	2	1	6
3	2	1			

Teorema.

Il problema Load Balancing è NPO Completo

Questo ci porta subito a studiare le soluzioni approssimate.

Inserire algoritmo qui che non ho sbatti, e anche i vari tempi e cazzate varie.

Teorema.

Greedy Load Balancing è 2 approssimato per Load Balancing.

Dimostrazione

Osservazione 1. $L^* \geq \frac{1}{m} \sum_j t_j$

Dim 1. $\sum_i L_i^* = \sum_j t_j$

Per pidgeonholing, non è possibile che $\forall i L_i^* < \frac{1}{m} \sum_j t_j$
quindi

$$\exists i L_i^* \geq \frac{1}{m} \sum_j t_j$$

Osservazione 2. $L^* \geq \max_j t_j$

Sia $\hat{i}$ la macchina più carica alla fine dell'esecuzione e $\hat{j}$ l'ultima task assegnata a $\hat{i}$.

Allora possiamo dire che:

$$L_{\hat{i}} - t_{\hat{j}} \leq L'_i \quad \forall i \in m \leq L_i$$

A questo punto sommiamo le disequazioni:

$$L_{\hat{i}} - t_{\hat{j}} \leq L'_i$$

$$m(L_{\hat{i}} - t_{\hat{j}}) \leq \sum_i L_i$$

$$L_{\hat{i}} - t_{\hat{j}} \leq \frac{1}{m} \sum_i L_i = \frac{1}{m} \sum_j t_j \leq L^*$$

$$L = L_{\hat{i}} = (L_{\hat{i}} - t_{\hat{j}}) + t_{\hat{j}} \leq L^* + t_{\hat{j}} \leq L^* + L^*$$

$$L \leq 2L^*$$

Corollario. Load Balancing $\in$ 2-APX

Teorema.

$\forall \epsilon > 0$ esiste un input in cui Greedy Load Balancing produce una soluzione di costo:

$$2 - \epsilon \leq \frac{L}{L^*} \leq 2$$

Dimostrazione

Prendiamo $m > \frac{1}{\epsilon}$, e $n = (m - 1) + 1$, con i primi $m - 1$ input pari a 1 e l'ultimo pari a m .

Applicando l'algoritmo su questi input otteniamo:

m	tasks	L_i			
	0 1 2				
0	<table><tr><td>1</td><td>1</td><td>1</td></tr></table>	1	1	1	13
1	1	1			
	0 1 2				
1	<table><tr><td>1</td><td>1</td><td>3</td></tr></table>	1	1	3	3
1	1	3			
	0 1				
2	<table><tr><td>1</td><td>1</td></tr></table>	1	1	3	
1	1				
$L = 2m - 1$					

Notiamo invece il caso ottimale su questo input:

m	tasks	L_i			
	0 1 2				
0	<table><tr><td>1</td><td>1</td><td>1</td></tr></table>	1	1	1	3
1	1	1			
	0 1 2				
1	<table><tr><td>1</td><td>1</td><td>1</td></tr></table>	1	1	1	3
1	1	1			
	0 1				
2	<table><tr><td>1</td><td>3</td></tr></table>	1	3	4	
1	3				
$L = 4$, ottimo					

Piccola digressione

L'input definito è inoltre uno dei pochi casi in cui la media equivale al caso ottimo.

$$\frac{m(m-1) + m}{m} = m - 1 + 1 = \lceil m \rceil$$

$$\frac{L}{L^*} = \frac{2m-1}{m} = 2 - \frac{1}{m} \geq 2 - \epsilon$$

Se ordino Greedy Load Balancing, perdo la sua proprietà di essere ONLINE, ma ne guadagno in approssimazione, infatti:

```

1  Sorto i task in ordine non crescente di durata
2  Applico Greedy Load Balancing

```

Listing 1: Offline Greedy Load Balancing

Teorema.

Offline Greedy Load Balancing è $\frac{3}{2}$ approssimato per Load Balancing.

Dimostrazione.

Osservazione 1. Se $n \leq m$, troviamo l'ottimo

Assumiamo quindi $n > m$

Osservazione 2. $L^* \geq 2t_m$ dato che

Dim 2.

$\underbrace{t_0 \geq t_1 \geq \dots \geq t_{m-1}}_{m \text{ task}} \geq t_m \dots$ sempre per pidgeonholing

$\exists i \ L^* \geq L_i^* \geq t_1 + t_2 \geq t_m + t_m$

quindi

$L^* \geq 2t_m$

Sia $\hat{i}$ la macchina più carica alla fine dell'esecuzione.

Possiamo assumere che ad essa siano assegnati almeno due task, altrimenti avremmo l'ottimo.

Sia $\hat{j}$ l'ultimo task assegnato:

$$\hat{j} \geq m$$

quindi

$$t_{\hat{j}} \leq t_m \leq \frac{1}{2}L^*$$

quindi

$$L = L_{\hat{i}} = L_{\hat{i}} - t_{\hat{j}} + t_{\hat{j}} \leq \frac{3}{2}L^*$$

N.B. 1: L'analisi è migliorabile, dimostrando che l'algoritmo è $\frac{4}{3}$ approssimato.
[Graham 1969]

N.B. 2: Load Balancing $\notin$ FPTAS (assunto $P \neq NP$)
[Hochbaum-Shmops 1988]